GEMS OUR OWN ENGLISH HIGH SCHOOL, SHARJAH

COMPUTER SCIENCE PROJECT

RevZone

SPORTS CAR SHOWROOM

Submitted by
KHADIJA SHIRAZ
GRADE 12 A

Guided by
MS. MINI

Academic Year - 2025-26

Our Own English High School, Sharjah

Certificate

This is to certify that _____

of class _____ Registration No. _____ has satisfactorily completed the project work in Computer Science during the academic year _____as prescribed by C. B. S. E, New Delhi, India.

Date of Examination: _____

_____ _____

Signature of Teacher-in-charge

Signature of External Examiner

ACKNOWLEGMENT

I would like to express my sincere gratitude to my Computer Science teacher, Ms. Mini for her guidance and support throughout this project.

I am also thankful to my parents for their encouragement and motivation, which helped me complete this project successfully.

Finally, I would like to acknowledge all the online resources and tutorials that assisted me in learning and implementing Python, Tkinter, and MySQL for this project.

Abstract

This project **“RevZone: Digital Sports Car Showroom”** is a desktop application built with **Python and Tkinter**, using **MySQL** for car data. It allows users to browse cars by type—**Electric, Hybrid, or Petrol**—view specifications, eco-performance details, and high-quality images.

The app includes **animated menus**, a **Wishlist** for saving favorite cars, and a **test drive booking system** with date and time options. It also highlights **sustainability** by showing fuel use, CO_2 emissions, recycled content, and an eco-score, linking the project to **UN SDG 12**.

This project demonstrates **programming** and **database management** skills while offering a digital showroom experience. **Future improvements** could include features such as **login** and **payment integration**.

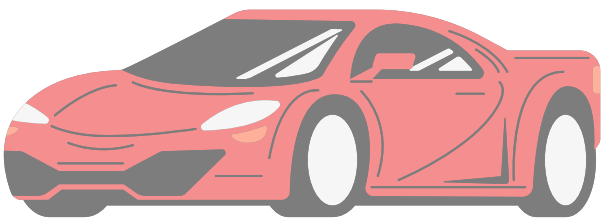


Table of CONTENTS

System Specification

a) Hardware Requirements

- Processor: Intel Core i3 / Apple Silicon (M1/M2) or above
- Operating System: Windows 10/11 or MacOS
- Memory (RAM): 4 GB minimum, 8 GB recommended
- Disk Capacity: 500 MB free space

b) Software Requirements

- Programming Language: Python 3.7+
- Database: MySQL 8.0
- Connector: mysql-connector-python
- GUI Toolkit: Tkinter
- Other Tools: PIL (Pillow library) for images

Project Overview

This project "**RevZone**: **Digital Sports Car Showroom**" is a desktop application that gives users a virtual showroom experience. It is made using **Python** with Tkinter for the interface and **MySQL** for storing car details and bookings. The aim is to make **exploring cars easy, modern, and eco-friendly.**

When the program starts, an **animated start page** is shown, followed by the **main menu**. From the menu, users can choose to **browse cars, open their wishlist, check test drive bookings, or exit.**

In the **Browse Cars section,** models are grouped into three categories: **Electric**, **Hybrid**, and **Petrol**. Each car displays its name, model, year, and type. Clicking a car shows more details like **engine, horsepower, speed, price, and an image.** Users also get the option to view the car **image** in **full screen**. **Eco-performance** data is shown as well, including **fuel consumption, CO_2 emissions, recycled content, and an eco-score.**

Users can **add cars** to a **wishlist**, which can be opened later to **view or remove** saved cars.

There is also a **Test Drive Booking feature** where users enter their **name, email, phone, and select a date and time.** All bookings are stored in the database and can be viewed and managed inside the app.

The project highlights **sustainable choices** by giving eco-data for each car and supports the **idea of responsible consumption, linking it with UN SDG 12**.

Overall, it combines programming, database handling, and design to provide a **complete digital showroom experience.**

Database Design

Table : CARS

Field	Type	Null	Key	Default	Extra
ID	int	NO	PRI	NULL	
BRAND	varchar(50)	YES		NULL	
MODEL	varchar(100)	YES		NULL	
TYPE	varchar(50)	YES		NULL	
YEAR	int	YES		NULL	
ENGINE	varchar(100)	YES		NULL	
HORSEPOWER	int	YES		NULL	
TOP_SPEED_KMPH	int	YES		NULL	
PRICE_USD	int	YES		NULL	
IMAGE_PATH	varchar(255)	YES		NULL	

Table : ECO_DATA

Field	Type	Null	Key	Default	Extra
CAR_ID	int	YES	MUL	NULL	
FUEL_CONSUMPTION_L_PER_100KM	float	YES		NULL	
RECYCLED_CONTENT_PCT	int	YES		NULL	
CO2_EMISSIONS_G_PER_KM	int	YES		NULL	
ECO_SCORE_PCT	int	YES		NULL	

Database Design

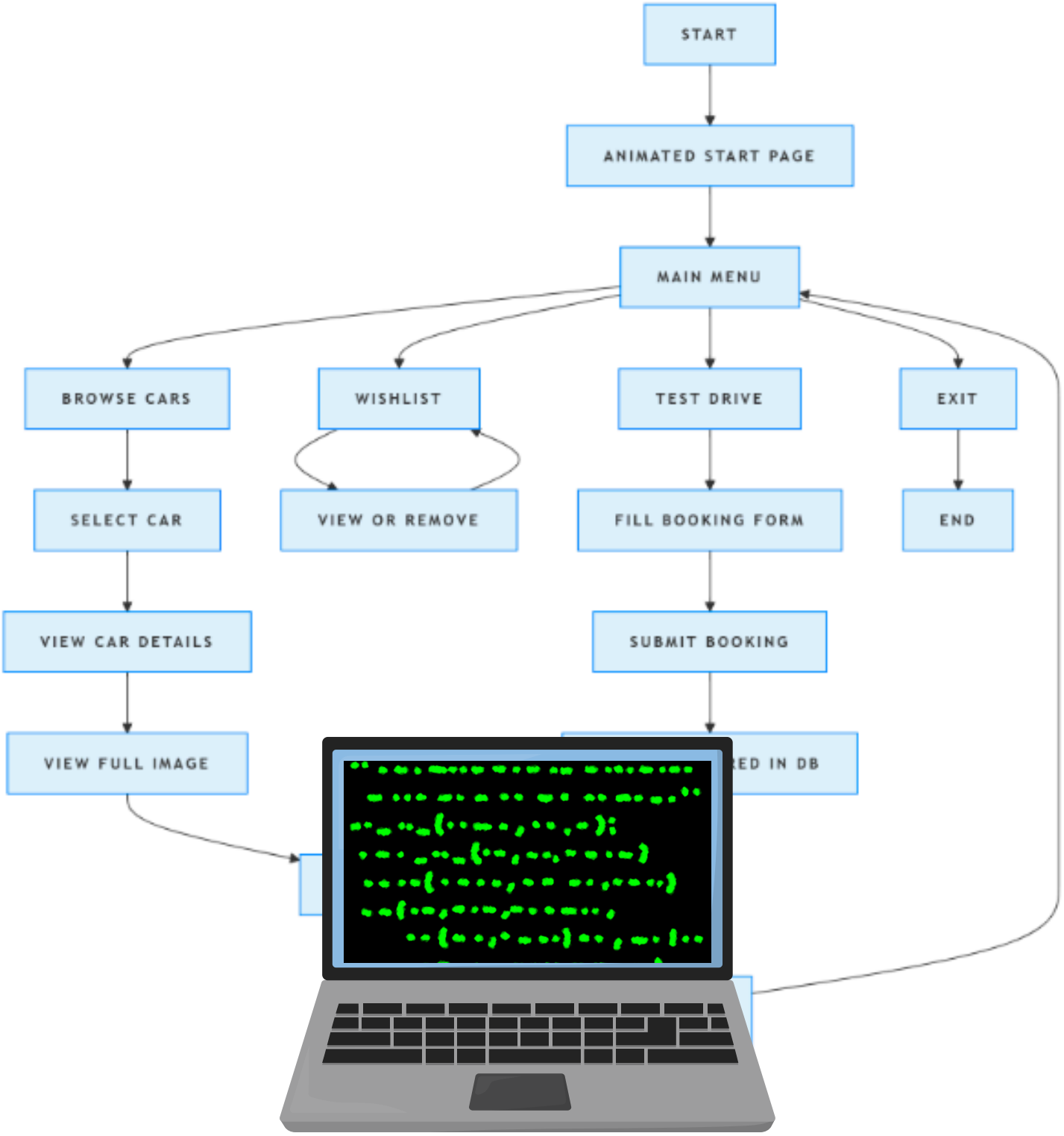
Table : WISHLIST

Field	Type	Null	Key	Default	Extra
ID	int	NO	PRI	NULL	auto_increment
CAR_ID	int	YES		NULL	
ADDED_ON	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

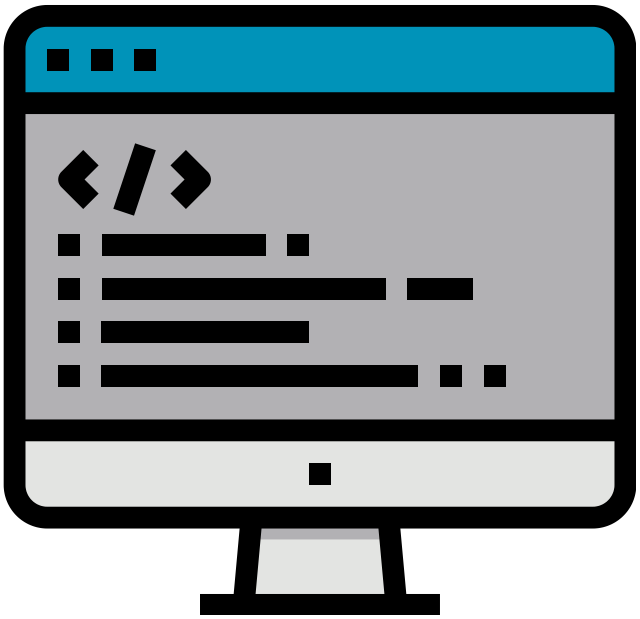
Table : TEST_DRIVES

Field	Type	Null	Key	Default	Extra
ID	int	NO	PRI	NULL	auto_increment
CAR_ID	int	YES	MUL	NULL	
FULL_NAME	varchar(100)	YES		NULL	
EMAIL	varchar(100)	YES		NULL	
PHONE	varchar(20)	YES		NULL	
PREFERRED_DATE	date	YES		NULL	
PREFERRED_TIME	time	YES		NULL	
STATUS	enum('PENDING','CONFIRMED','CANCELLED')	YES		PENDING	
CREATED_AT	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

Flowchart


START
ANIMATED START PAGE
MAIN MENU
BROWSE CARS
WISHLIST
TEST DRIVE
EXIT
SELECT CAR
VIEW OR REMOVE
FILL BOOKING FORM
END
VIEW CAR DETAILS
SUBMIT BOOKING
VIEW FULL IMAGE

Program Source Code



```
from tkinter import *
from tkinter import ttk
from tkinter import messagebox
from PIL import Image, ImageTk
import mysql.connector as m

db = m.connect(host='localhost', user='root', password='1234', database='showroom')
cur = db.cursor()

window = Tk()
window.title('RevZone')
window.attributes('-fullscreen', True)
window.bind('<Escape>', lambda e: window.attributes('-fullscreen', False))
window.config(bg='black')

screen_width = window.winfo_screenwidth()
screen_height = window.winfo_screenheight()

app_started = False

img = Image.open(r"C:\Users\Farooqui\Downloads\Khadija\CS Project\Images\Start Pg Car.jpeg")
img = img.resize((screen_width, screen_height))
photo = ImageTk.PhotoImage(img)

canvas = Canvas(window, width=screen_width, height=screen_height, highlightthickness=0)
canvas.pack()
canvas.create_image(0, 0, image=photo, anchor=NW)

text_size = 5
text_obj = canvas.create_text(1100, 560, text='RevZone', font=('Impact', text_size),
fill='ghostwhite')
canvas.create_text(1100, 630, text='Zone In, Rev Out!', font=('Chiller', 30, 'bold italic'),
fill='white')

animation_running = True

def animate_text():
    global text_size, animation_running
    if text_size < 60 and animation_running:
        text_size += 2
        try:
            canvas.itemconfig(text_obj, font=('Impact', text_size))
            window.after(10, animate_text)
        except:
            animation_running = False

def on_closing():
    global animation_running
    animation_running = False
    window.destroy()

window.protocol("WM_DELETE_WINDOW", on_closing)

animate_text()

def create_styled_button(parent, text, command, bg_color=None, hover_color=None, width=25):
    if bg_color is None:
        bg_color = '#dc143c'
    if hover_color is None:
        hover_color = '#b22222'

    btn = Button(parent, text=text, font=('Orbitron', 14, 'bold'),
                 bg=bg_color, fg='white',
                 bd=0, pady=12, width=width,
                 cursor='hand2', command=command,
                 relief=FLAT, activebackground=hover_color,
                 activeforeground='white')

```

```
    def on_enter(e):
        btn.config(bg=hover_color)
    def on_leave(e):
        btn.config(bg=bg_color)

    btn.bind("<Enter>", on_enter)
    btn.bind("<Leave>", on_leave)
    return btn

def view_wishlist():
    wl_window = Toplevel(window)
    wl_window.title("Wishlist")
    wl_window.geometry("1000x700")
    wl_window.config(bg='#0a0a0a')
    wl_window.resizable(True, True)

    header_frame = Frame(wl_window, bg='#2a2a2a', height=80)
    header_frame.pack(fill=X, pady=(0,20))
    header_frame.pack_propagate(False)

    Label(header_frame, text="⭐ MY WISHLIST",
          font=('Orbitron', 25, 'bold'),
          fg='white', bg='#2a2a2a').pack(pady=20)

    main_frame = Frame(wl_window, bg='#0a0a0a')
    main_frame.pack(fill=BOTH, expand=True, padx=20, pady=10)

    cur.execute("SELECT CARS.ID, BRAND, MODEL FROM CARS JOIN WISHLIST ON CARS.ID =
WISHLIST.CAR_ID")
    items = cur.fetchall()

    if not items:
        empty_frame = Frame(main_frame, bg='#2a2a2a', relief=RIDGE, bd=2)
        empty_frame.pack(fill=X, pady=20, padx=20)
        Label(empty_frame, text="🚗 Your wishlist is empty",
              font=('Roboto', 18), fg='#cccccc',
              bg='#2a2a2a').pack(pady=40)
        Label(empty_frame, text="Add some dream cars to get started!",
              font=('Roboto', 14), fg='#c0c0c0',
              bg='#2a2a2a').pack(pady=(0,20))
    else:
        for i, (car_id, brand, model) in enumerate(items):
            card_frame = Frame(main_frame, bg='#2a2a2a', relief=RIDGE, bd=1)
            card_frame.pack(fill=X, pady=8, padx=10)

            content_frame = Frame(card_frame, bg='#2a2a2a')
            content_frame.pack(fill=X, pady=15, padx=20)

            car_label = Label(content_frame, text=f"🏎️ {brand} {model}",
                              font=('Roboto', 16, 'bold'),
                              fg='white', bg='#2a2a2a')
            car_label.pack(side=LEFT, anchor=W)

            remove_btn = Button(content_frame, text="✕ REMOVE",
                                font=('Roboto', 10, 'bold'),
                                bg='#dc143c', fg='white',
                                bd=0, pady=8, padx=15, cursor='hand2',
                                command=lambda cid=car_id, fr=card_frame:
remove_from_db_wishlist(cid, fr))
            remove_btn.pack(side=RIGHT, padx=(10,0))

    create_styled_button(wl_window, "← BACK TO MENU", wl_window.destroy,
                         '#2a2a2a', '#dc143c').pack(pady=20)

def remove_from_db_wishlist(car_id, frame):
    cur.execute("DELETE FROM WISHLIST WHERE CAR_ID = %s", (car_id,))
    db.commit()
    frame.destroy()

```

```
def start_app():
    car_window = Toplevel(window)
    car_window.title("Browse Cars")
    car_window.attributes('-fullscreen', True)
    car_window.bind('<Escape>', lambda e: car_window.destroy())
    car_window.config(bg='#0a0a0a')

    main_container = Frame(car_window, bg='#0a0a0a')
    main_container.pack(fill=BOTH, expand=True, padx=20, pady=20)

    header = Frame(main_container, bg='#0a0a0a')
    header.pack(fill=X, pady=(0, 20))

    Label(header, text="Car Models",
          font=("Impact", 28), fg='white', bg='#0a0a0a').pack(side=LEFT)

    Button(header, text="← Back to Menu", font=("Arial", 12, "bold"),
           bg="dimgray", fg="white", command=car_window.destroy).pack(side=RIGHT)

    content = Frame(main_container, bg='#0a0a0a')
    content.pack(fill=BOTH, expand=True)

    green_zone = Frame(content, bg='#0a0a0a')
    green_zone.pack(side=LEFT, fill=BOTH, expand=True, padx=(0, 10))

    green_header = Frame(green_zone, bg='#1a3c2d', relief=RAISED, bd=1)
    green_header.pack(fill=X, pady=(0, 10))
    Label(green_header, text="🌿 Green Zone (Eco-Friendly)",
          font=("Impact", 22), fg="white", bg='#1a3c2d').pack(pady=15)

    green_container = Frame(green_zone, bg='#0a0a0a')
    green_container.pack(fill=BOTH, expand=True)
    green_container.grid_rowconfigure(0, weight=1)
    green_container.grid_rowconfigure(1, weight=1)
    green_container.grid_columnconfigure(0, weight=1)

    electric_frame = Frame(green_container, bg='#2a4c3d', relief=RAISED, bd=1)
    electric_frame.grid(row=0, column=0, sticky="nsew", pady=(0, 5))

    electric_header = Frame(electric_frame, bg='#2a4c3d')
    electric_header.pack(fill=X)
    Label(electric_header, text="🔋 Electric Vehicles",
          font=("Impact", 18), fg="white", bg='#2a4c3d').pack(side=LEFT, padx=20, pady=10)
    Label(electric_header, text="Zero Emissions • Sustainable Future",
          font=("Arial", 12, "italic"), fg='#00cc66', bg='#2a4c3d').pack(side=RIGHT, padx=20)

    hybrid_frame = Frame(green_container, bg='#2a4c5d', relief=RAISED, bd=1)
    hybrid_frame.grid(row=1, column=0, sticky="nsew", pady=(5, 0))

    hybrid_header = Frame(hybrid_frame, bg='#2a4c5d')
    hybrid_header.pack(fill=X)
    Label(hybrid_header, text="⚡ Hybrid Vehicles",
          font=("Impact", 18), fg="white", bg='#2a4c5d').pack(side=LEFT, padx=20, pady=10)
    Label(hybrid_header, text="Reduced Emissions • Better Efficiency",
          font=("Arial", 12, "italic"), fg='#3399ff', bg='#2a4c5d').pack(side=RIGHT, padx=20)

    regular_zone = Frame(content, bg='#0a0a0a')
    regular_zone.pack(side=RIGHT, fill=BOTH, expand=True, padx=(10, 0))

    regular_header = Frame(regular_zone, bg='#3c2a2a', relief=RAISED, bd=1)
    regular_header.pack(fill=X, pady=(0, 10))
    Label(regular_header, text="🚙 Performance Zone",
          font=("Impact", 22), fg="white", bg='#3c2a2a').pack(pady=15)
    Label(regular_header, text="Consider eco-friendly options for sustainability",
          font=("Arial", 10, "italic"), fg="silver", bg='#3c2a2a').pack(pady=(0, 10))

    regular_container = Frame(regular_zone, bg='#4c3a3a', relief=RAISED, bd=1)
    regular_container.pack(fill=BOTH, expand=True)

```

```
    cur.execute("SELECT ID, BRAND, MODEL, TYPE, YEAR, IMAGE_PATH FROM CARS")
    cars = cur.fetchall()

    def create_scrollable_frame(parent, bg_color):
        canvas = Canvas(parent, bg=bg_color, highlightthickness=0)
        scrollbar = Scrollbar(parent, orient="vertical", command=canvas.yview)
        scrollable_frame = Frame(canvas, bg=bg_color)

        scrollable_frame.bind(
            "<Configure>",
            lambda e: canvas.configure(scrollregion=canvas.bbox("all"))
        )

        canvas.create_window((0, 0), window=scrollable_frame, anchor="nw")
        canvas.configure(yscrollcommand=scrollbar.set)

        canvas.pack(side="left", fill="both", expand=True)
        scrollbar.pack(side="right", fill="y")

        return scrollable_frame

    electric_scroll = create_scrollable_frame(electric_frame, '#2a4c3d')
    hybrid_scroll = create_scrollable_frame(hybrid_frame, '#2a4c5d')
    regular_scroll = create_scrollable_frame(regular_container, '#4c3a3a')

    for car in cars:
        car_id, brand, model, car_type, year, image_path = car

        def on_click(event, car_id=car_id):
            show_car_details(car_id)

        if car_type.lower() == "electric":
            parent = electric_scroll
            bg_color = '#3a5c4d'
            text_color = '#00cc66'
        elif car_type.lower() in ("hybrid", "plug-in hybrid"):
            parent = hybrid_scroll
            bg_color = '#3a5c6d'
            text_color = '#3399ff'
        else:
            parent = regular_scroll
            bg_color = '#5c4a4a'
            text_color = '#ff3366'

        car_frame = Frame(parent, bg=bg_color, relief=RAISED, bd=1)
        car_frame.pack(fill=X, pady=5, padx=10)

        car_label = Label(car_frame, text=f"{brand} {model} ({year})",
                          font=("Arial", 14, "bold"), fg="white", bg=bg_color, cursor="hand2")
        car_label.pack(anchor='w', pady=10, padx=20)
        car_label.bind("<Button-1>", on_click)

        type_label = Label(car_frame, text=car_type.upper(),
                           font=("Arial", 10), fg=text_color, bg=bg_color)
        type_label.pack(anchor='w', padx=20, pady=(0, 10))

def show_car_details(car_id):
    details_window = Toplevel(window)
    details_window.title("Car Details")
    details_window.attributes('-fullscreen', True)
    details_window.bind('<Escape>', lambda e: details_window.destroy())
    details_window.config(bg='#0a0a0a')

    cur.execute("SELECT BRAND, MODEL, TYPE, YEAR, ENGINE, HORSEPOWER, TOP_SPEED_KMPH, PRICE_USD,
IMAGE_PATH FROM CARS WHERE ID = %s", (car_id,))
    car = cur.fetchone()

    cur.execute("SELECT FUEL_CONSUMPTION_L_PER_100KM, RECYCLED_CONTENT_PCT,
CO2_EMISSIONS_G_PER_KM, ECO_SCORE_PCT FROM ECO_DATA WHERE CAR_ID = %s", (car_id,))
```

```
    eco_data = cur.fetchone()

    if car:
        brand, model, type, year, engine, horsepower, speed, price, image_path = car

        Button(details_window, text="← Back", font=('Arial', 12),
               bg='gray', fg='white', command=details_window.destroy,
               bd=0, padx=20, pady=5).pack(anchor='nw', padx=20, pady=20)

        title_frame = Frame(details_window, bg='#0a0a0a')
        title_frame.pack(fill=X, pady=(0,30))

        Label(title_frame, text=f"{brand} {model}",
              font=('Arial', 36, 'bold'),
              fg='white', bg='#0a0a0a').pack(side=LEFT, padx=(40,20))

        if type.lower() == "electric":
            badge_color = '#228B22'
            badge_text = "ELECTRIC"
        elif type.lower() in ("hybrid", "plug-in hybrid"):
            badge_color = '#4d79ff'
            badge_text = "HYBRID"
        else:
            badge_color = '#dc143c'
            badge_text = "PETROL"

        type_badge = Label(title_frame, text=badge_text,
                           font=('Arial', 12, 'bold'),
                           fg='white', bg=badge_color,
                           padx=15, pady=5)
        type_badge.pack(side=RIGHT, padx=(20,40), anchor='ne')

        main_content = Frame(details_window, bg='#0a0a0a')
        main_content.pack(fill=BOTH, expand=True, padx=40)

        left_content = Frame(main_content, bg='#0a0a0a')
        left_content.pack(side=LEFT, fill=BOTH, expand=True, padx=(0,20))

        spec_frame = Frame(left_content, bg='#2a2a2a', relief=RIDGE, bd=1)
        spec_frame.pack(fill=X, pady=(0,20))

        Label(spec_frame, text="Specifications",
              font=('Arial', 20, 'bold'),
              fg='#4da6ff', bg='#2a2a2a').pack(pady=(15,10), padx=20, anchor='w')

        specs = [
            ("Type:", type),
            ("Year:", year),
            ("Engine:", engine),
            ("Horsepower:", horsepower),
            ("Top Speed (km/h):", speed),
            ("Price (USD):", f"${price:,.2f}")
        ]

        for label_text, value in specs:
            spec_row = Frame(spec_frame, bg='#2a2a2a')
            spec_row.pack(fill=X, pady=3, padx=20)

            Label(spec_row, text=label_text,
                  font=('Arial', 14),
                  fg='#cccccc', bg='#2a2a2a').pack(side=LEFT)
            Label(spec_row, text=str(value),
                  font=('Arial', 14),
                  fg='white', bg='#2a2a2a').pack(side=RIGHT)

        Label(spec_frame, text="", bg='#2a2a2a').pack(pady=10)

        if eco_data:
            eco_frame = Frame(left_content, bg='#2a2a2a', relief=RIDGE, bd=1)
```

```
            eco_frame.pack(fill=X)

            Label(eco_frame, text="Eco Performance",
                  font=('Arial', 20, 'bold'),
                  fg='#00ff99', bg='#2a2a2a').pack(pady=(15,10), padx=20, anchor='w')

            fuel_cons, recycled, co2, eco_score = eco_data
            eco_specs = [
                ("Fuel Consumption (L/100km):", fuel_cons),
                ("Recycled Content (%):", recycled),
                ("CO2 Emissions (g/km):", co2),
                ("Eco Score (%):", eco_score)
            ]

            for label_text, value in eco_specs:
                eco_row = Frame(eco_frame, bg='#2a2a2a')
                eco_row.pack(fill=X, pady=3, padx=20)

                Label(eco_row, text=label_text,
                      font=('Arial', 14),
                      fg='#cccccc', bg='#2a2a2a').pack(side=LEFT)
                Label(eco_row, text=str(value),
                      font=('Arial', 14),
                      fg='white', bg='#2a2a2a').pack(side=RIGHT)

            Label(eco_frame, text="", bg='#2a2a2a').pack(pady=10)

        right_content = Frame(main_content, bg='#0a0a0a', width=400)
        right_content.pack(side=RIGHT, fill=Y, padx=(20,0))
        right_content.pack_propagate(False)

        img_frame = Frame(right_content, bg='#2a2a2a', relief=RIDGE, bd=1, height=300)
        img_frame.pack(fill=X, pady=(0,20))
        img_frame.pack_propagate(False)

        try:
            img = Image.open(image_path)
            img = img.resize((380, 280), Image.LANCZOS)
            img_tk = ImageTk.PhotoImage(img)
            img_label = Label(img_frame, image=img_tk, bg='#2a2a2a')
            img_label.image = img_tk
            img_label.pack(expand=True, pady=10)
        except:
            Label(img_frame, text="Image Preview\nNot Available",
                  font=('Arial', 16), fg='#cccccc',
                  bg='#2a2a2a').pack(expand=True)

        def open_image():
            try:
                img_window = Toplevel(window)
                img_window.title(f"{brand} {model} - Image")
                img_window.attributes('-fullscreen', True)
                img_window.bind('<Escape>', lambda e: img_window.destroy())
                img_window.config(bg='#0a0a0a')

                back_button = Button(img_window, text="← BACK",
                                     font=('Orbitron', 14, 'bold'),
                                     bg='#555555', fg='white',
                                     command=img_window.destroy,
                                     bd=0, padx=20, pady=10, cursor='hand2',
                                     relief=FLAT, activebackground='#777777',
                                     activeforeground='white')

                back_button.place(x=20, y=20)
                back_button.lift()

                def on_enter_back(e):
                    back_button.config(bg='#777777')
                def on_leave_back(e):
```

```
                    back_button.config(bg='#555555')
                back_button.bind("<Enter>", on_enter_back)
                back_button.bind("<Leave>", on_leave_back)

                img = Image.open(image_path)
                screen_w = img_window.winfo_screenwidth()
                screen_h = img_window.winfo_screenheight()
                img_ratio = img.width / img.height
                screen_ratio = screen_w / screen_h

                if img_ratio > screen_ratio:
                    new_width = screen_w
                    new_height = int(screen_w / img_ratio)
                else:
                    new_height = screen_h
                    new_width = int(screen_h * img_ratio)

                img = img.resize((new_width, new_height), Image.LANCZOS)
                img_tk = ImageTk.PhotoImage(img)

                img_label = Label(img_window, image=img_tk, bg='#0a0a0a')
                img_label.image = img_tk
                img_label.place(relx=0.5, rely=0.5, anchor="center")

                back_button.lift()

            except Exception as e:
                print(f"Error opening image: {e}")
                error_label = Label(details_window, text="❌ Image not found",
                                    font=('Arial', 14),
                                    fg='#dc143c', bg='#0a0a0a')
                error_label.pack(pady=10)

        def add_to_wishlist():
            try:
                if not hasattr(right_content, 'message_frame'):
                    right_content.message_frame = Frame(right_content, bg='#0a0a0a', height=30)
                    right_content.message_frame.pack(fill=X, pady=(0, 10), after=test_drive_btn)

                for widget in right_content.message_frame.winfo_children():
                    widget.destroy()

                cur.execute("SELECT * FROM WISHLIST WHERE CAR_ID = %s", (car_id,))
                if cur.fetchone():
                    message_label = Label(right_content.message_frame, text="⚠ Already in
Wishlist!",
                                          font=('Arial', 12, 'bold'),
                                          fg='#dc143c', bg='#0a0a0a')
                    message_label.pack(pady=5)
                    details_window.after(2000, lambda: message_label.destroy() if
message_label.winfo_exists() else None)
                else:
                    cur.execute("INSERT INTO WISHLIST (CAR_ID) VALUES (%s)", (car_id,))
                    db.commit()
                    message_label = Label(right_content.message_frame, text="✅ Added to
Wishlist!",
                                          font=('Arial', 12, 'bold'),
                                          fg='#00ff99', bg='#0a0a0a')
                    message_label.pack(pady=5)
                    details_window.after(2000, lambda: message_label.destroy() if
message_label.winfo_exists() else None)
            except Exception as e:
                print(f"Error: {e}")
                if not hasattr(right_content, 'message_frame'):
                    right_content.message_frame = Frame(right_content, bg='#0a0a0a', height=30)
                    right_content.message_frame.pack(fill=X, pady=(0, 10), before=view_img_btn)

                for widget in right_content.message_frame.winfo_children():
                    widget.destroy()
```

```
                message_label = Label(right_content.message_frame, text="⚠ Error adding to
Wishlist",
                                      font=('Arial', 12),
                                      fg='#dc143c', bg='#0a0a0a')
                message_label.pack(pady=5)
                details_window.after(2000, lambda: message_label.destroy() if
message_label.winfo_exists() else None)

        view_img_btn = Button(right_content, text="🖼 VIEW FULL IMAGE",
                              font=('Orbitron', 14, 'bold'),
                              bg='lightblue', fg='white',
                              bd=0, pady=10, cursor='hand2',
                              command=open_image,
                              relief=FLAT, activebackground='#1a73e8',
                              activeforeground='white')
        view_img_btn.pack(fill=X, pady=(0, 10), padx=5)

        def on_enter_view(e):
            view_img_btn.config(bg='#1a73e8')
        def on_leave_view(e):
            view_img_btn.config(bg='lightblue')
        view_img_btn.bind("<Enter>", on_enter_view)
        view_img_btn.bind("<Leave>", on_leave_view)

        wishlist_btn = Button(right_content, text="⭐ ADD TO WISHLIST",
                              font=('Orbitron', 14, 'bold'),
                              bg='deepskyblue', fg='white',
                              bd=0, pady=10, cursor='hand2',
                              command=add_to_wishlist,
                              relief=FLAT, activebackground='#0d47a1',
                              activeforeground='white')
        wishlist_btn.pack(fill=X, pady=(0, 10), padx=5)

        def on_enter_wish(e):
            wishlist_btn.config(bg='#0d47a1')
        def on_leave_wish(e):
            wishlist_btn.config(bg='deepskyblue')
        wishlist_btn.bind("<Enter>", on_enter_wish)
        wishlist_btn.bind("<Leave>", on_leave_wish)

        test_drive_btn = Button(right_content, text="🚙 BOOK TEST DRIVE",
                                font=('Orbitron', 14, 'bold'),
                                bg='navy', fg='white',
                                bd=0, pady=10, cursor='hand2',
                                command=lambda: book_test_drive(car_id),
                                relief=FLAT, activebackground='#1a73e8',
                                activeforeground='white')
        test_drive_btn.pack(fill=X, pady=(0, 10), padx=5)

        def on_enter_test(e):
            test_drive_btn.config(bg='#1a73e8')
        def on_leave_test(e):
            test_drive_btn.config(bg='navy')
        test_drive_btn.bind("<Enter>", on_enter_test)
        test_drive_btn.bind("<Leave>", on_leave_test)

def book_test_drive(car_id):
    booking_window = Toplevel(window)
    booking_window.title("Book Test Drive")
    booking_window.geometry("500x700")
    booking_window.config(bg='#0a0a0a')
    booking_window.resizable(False, False)

    booking_window.transient(window)
    booking_window.grab_set()

    header_frame = Frame(booking_window, bg='#2a2a2a', height=80)
    header_frame.pack(fill=X, pady=(0, 20))
```

```
    header_frame.pack_propagate(False)

    Label(header_frame, text="🚙 BOOK TEST DRIVE",
          font=('Orbitron', 18, 'bold'), fg='#ffffff', bg='#2a2a2a').pack(pady=20)

    cur.execute("SELECT BRAND, MODEL FROM CARS WHERE ID = %s", (car_id,))
    car = cur.fetchone()
    brand, model = car if car else ("Unknown", "Car")

    car_info_frame = Frame(booking_window, bg='#2a2a2a', relief=RIDGE, bd=1)
    car_info_frame.pack(fill=X, pady=(0, 20), padx=20)
    Label(car_info_frame, text=f"{brand} {model}",
          font=('Arial', 16, 'bold'),
          fg='#ffffff', bg='#2a2a2a').pack(pady=15)

    canvas = Canvas(booking_window, bg='#0a0a0a', highlightthickness=0)
    scrollbar = Scrollbar(booking_window, orient="vertical", command=canvas.yview)
    scrollable_frame = Frame(canvas, bg='#0a0a0a')

    scrollable_frame.bind(
        "<Configure>",
        lambda e: canvas.configure(scrollregion=canvas.bbox("all"))
    )

    canvas.create_window((0, 0), window=scrollable_frame, anchor="nw")
    canvas.configure(yscrollcommand=scrollbar.set)

    canvas.pack(side="left", fill="both", expand=True, padx=20)
    scrollbar.pack(side="right", fill="y")

    form_frame = scrollable_frame
    text_fields = [
        ("Full Name", "text"),
        ("Email", "email"),
        ("Phone", "tel")]

    entries = {}
    for field_name, field_type in text_fields:
        Label(form_frame, text=field_name,
              font=('Arial', 12),
              fg='#cccccc', bg='#0a0a0a').pack(anchor='w', pady=(10, 5))

        entry = Entry(form_frame, font=('Arial', 12),
                      bg='#2a2a2a', fg='#ffffff',
                      insertbackground='white', bd=1, relief=SOLID)
        entry.pack(fill=X, pady=(0, 10))
        entries[field_name] = entry

    Label(form_frame, text="Preferred Date",
          font=('Arial', 12),
          fg='#cccccc', bg='#0a0a0a').pack(anchor='w', pady=(10, 5))

    from datetime import datetime, timedelta
    dates = []
    for i in range(7):
        date = datetime.now() + timedelta(days=i+1)
        dates.append(date.strftime("%Y-%m-%d"))

    date_var = StringVar()
    date_dropdown = ttk.Combobox(form_frame, textvariable=date_var,
                                 values=dates, state="readonly",
                                 font=('Arial', 12))
    date_dropdown.pack(fill=X, pady=(0, 10))
    entries["Preferred Date"] = date_var

    Label(form_frame, text="Preferred Time",
          font=('Arial', 12),
          fg='#cccccc', bg='#0a0a0a').pack(anchor='w', pady=(10, 5))
```

```
    time_slots = []
    for hour in range(9, 18):
        for minute in [0, 30]:
            time_slots.append(f"{hour:02d}:{minute:02d}")

    time_var = StringVar()
    time_dropdown = ttk.Combobox(form_frame, textvariable=time_var,
                                 values=time_slots, state="readonly",
                                 font=('Arial', 12))
    time_dropdown.pack(fill=X, pady=(0, 10))
    entries["Preferred Time"] = time_var

    def submit_booking():
        try:
            full_name = entries["Full Name"].get()
            email = entries["Email"].get()
            phone = entries["Phone"].get()
            preferred_date = entries["Preferred Date"].get()
            preferred_time = entries["Preferred Time"].get()

            if not all([full_name, email, phone, preferred_date, preferred_time]):
                message_label.config(text="Please fill all fields!", fg='#dc143c')
                return

            cur.execute("""
                INSERT INTO test_drives (car_id, full_name, email, phone, preferred_date,
preferred_time)
                VALUES (%s, %s, %s, %s, %s, %s)
            """, (car_id, full_name, email, phone, preferred_date, preferred_time))
            db.commit()

            message_label.config(text="✅ Test drive booked successfully!", fg='#228B22')

            for key, entry in entries.items():
                if isinstance(entry, Entry):
                    entry.delete(0, END)
                else:
                    entry.set("")

            booking_window.after(2000, booking_window.destroy)

        except Exception as e:
            message_label.config(text=f"Error: {str(e)}", fg='#dc143c')

    message_label = Label(form_frame, text="", font=('Arial', 12),
                          bg='#0a0a0a')
    message_label.pack(pady=10)

    submit_btn = Button(form_frame, text="🗓️ Submit",
                        font=('Orbitron', 12, 'bold'),
                        bg='#1a73e8', fg='white',
                        bd=0, pady=12, cursor='hand2',
                        command=submit_booking,
                        relief=FLAT, activebackground='#0d47a1',
                        activeforeground='white')
    submit_btn.pack(fill=X, pady=20)

    def on_enter_submit(e):
        submit_btn.config(bg='#0d47a1')
    def on_leave_submit(e):
        submit_btn.config(bg='#1a73e8')
    submit_btn.bind("<Enter>", on_enter_submit)
    submit_btn.bind("<Leave>", on_leave_submit)

    cancel_btn = Button(form_frame, text="CANCEL",
                        font=('Arial', 12),
                        bg='#555555', fg='white',
                        bd=0, pady=8, cursor='hand2',
                        command=booking_window.destroy,
```

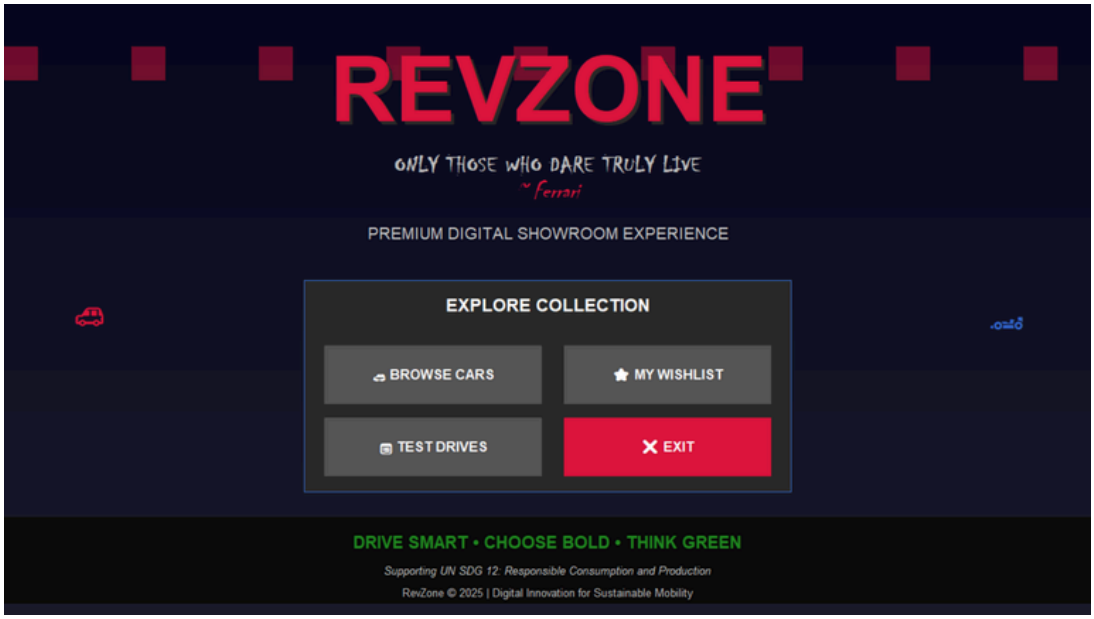
```
                        relief=FLAT, activebackground='#777777',
                        activeforeground='white')
    cancel_btn.pack(fill=X)

    def on_enter_cancel(e):
        cancel_btn.config(bg='#777777')
    def on_leave_cancel(e):
        cancel_btn.config(bg='#555555')
    cancel_btn.bind("<Enter>", on_enter_cancel)
    cancel_btn.bind("<Leave>", on_leave_cancel)


def view_test_drives():
    drives_window = Toplevel(window)
    drives_window.title("Test Drive Bookings")
    drives_window.geometry("1200x700")
    drives_window.config(bg='#0a0a0a')

    header_frame = Frame(drives_window, bg='#2a2a2a', height=80)
    header_frame.pack(fill=X, pady=(0, 20))
    header_frame.pack_propagate(False)

    Label(header_frame, text="🗓️ TEST DRIVE BOOKINGS",
          font=('Orbitron', 20, 'bold'),
          fg='#ffffff', bg='#2a2a2a').pack(pady=20)

    def refresh_tree():
        for item in tree.get_children():
            tree.delete(item)

        cur.execute("""SELECT td.id, c.brand, c.model, td.full_name, td.email, td.phone,
td.preferred_date, td.preferred_time, td.status
            FROM test_drives td
            JOIN cars c ON td.car_id = c.id
            ORDER BY td.created_at DESC""")
        drives = cur.fetchall()

        for drive in drives:
            car_info = f"{drive[1]} {drive[2]}"
            tree.insert("", "end", values=(
                drive[0], car_info, drive[3], drive[4], drive[5],
                drive[6], drive[7], drive[8]))

    def delete_booking():
        selected_item = tree.selection()
        if not selected_item:
            message_label.config(text="Please select a booking to delete", fg='#dc143c')
            return

        row_values = tree.item(selected_item[0])['values']

        booking_id = row_values[0]

        confirm = messagebox.askyesno("Confirm Delete",
                                      "Are you sure you want to delete this test drive booking?")
        if confirm:
            try:
                cur.execute("DELETE FROM test_drives WHERE id = %s", (booking_id,))
                db.commit()
                message_label.config(text="✅ Booking deleted successfully!", fg='#228B22')
                refresh_tree()
                drives_window.after(2000, lambda: message_label.config(text=""))
            except Exception as e:
                message_label.config(text=f"Error: {str(e)}", fg='#dc143c')

    content_frame = Frame(drives_window, bg='#0a0a0a')
    content_frame.pack(fill=BOTH, expand=True, padx=20, pady=10)

    action_frame = Frame(content_frame, bg='#0a0a0a')
```

```
814
815     for i in range(0, screen_height, 2):
816         shade = int(8 + (i / screen_height) * 20)
817         blue_shade = min(30 + int(i / screen_height * 15), 45)
818         color = f"#{shade:02x}{shade:02x}{blue_shade:02x}"
819         bg_canvas.create_rectangle(0, i, screen_width, i+2, fill=color, outline=color)
820
821     for i in range(0, screen_width, 150):
822         size = 40
823         rect_color = bg_canvas.create_rectangle(i, 50, i+size, 50+size,
824                                                 fill='#dc143c', outline='')
825         bg_canvas.itemconfig(rect_color, stipple="gray50")
826
827     bg_canvas.create_text(screen_width//2 + 4, 104, text="REVZONE",
828                           font=('Orbitron', 80, 'bold'),
829                           fill='#2a2a2a',
830                           justify=CENTER)
831
832     bg_canvas.create_text(screen_width//2, 100, text="REVZONE",
833                           font=('Orbitron', 80, 'bold'),
834                           fill='#dc143c',
835                           justify=CENTER)
836
837     bg_canvas.create_text(screen_width//2, 190,
838                           text="ONLY THOSE WHO DARE TRULY LIVE",
839                           font=('Chiller', 20, 'bold'),
840                           fill='#c0c0c0',
841                           justify=CENTER)
842
843     bg_canvas.create_text(screen_width//2, 220,
844                           text="~ Ferrari",
845                           font=('Chiller', 22, 'italic'),
846                           fill='#dc143c',
847                           justify=CENTER)
848
849     bg_canvas.create_text(screen_width//2, 270,
850                           text="PREMIUM DIGITAL SHOWROOM EXPERIENCE",
851                           font=('Poppins', 14),
852                           fill='#cccccc',
853                           justify=CENTER)
854
855     btn_container = Frame(bg_canvas, bg='#1a1a1a', relief=FLAT, bd=0)
856     btn_container.place(relx=0.5, rely=0.45, anchor="n", relwidth=0.45, relheight=0.35)
857
858     btn_bg = Frame(btn_container, bg='#2a2a2a',
859                    relief=FLAT, bd=0,
860                    highlightbackground='#3366cc',
861                    highlightthickness=1)
862     btn_bg.pack(fill=BOTH, expand=True, padx=1, pady=1)
863
864     Label(btn_bg, text="EXPLORE COLLECTION",
865           font=('Orbitron', 16, 'bold'),
866           fg='#ffffff', bg='#2a2a2a',
867           pady=15).pack()
868
869     button_grid = Frame(btn_bg, bg='#2a2a2a')
870     button_grid.pack(expand=True, fill=BOTH, padx=10, pady=10)
871
872     buttons_data = [
873         ("🚗", "BROWSE CARS", start_app, 0, 0),
874         ("⭐", "MY WISHLIST", view_wishlist, 0, 1),
875         ("🗓️", "TEST DRIVES", view_test_drives, 1, 0),
876         ("❌", "EXIT", window.destroy, 1, 1)
877     ]
878
879     for icon, text, command, row, col in buttons_data:
880         btn_frame = Frame(button_grid, bg='#2a2a2a')
881         btn_frame.grid(row=row, column=col, padx=8, pady=8, sticky="nsew")
882
```

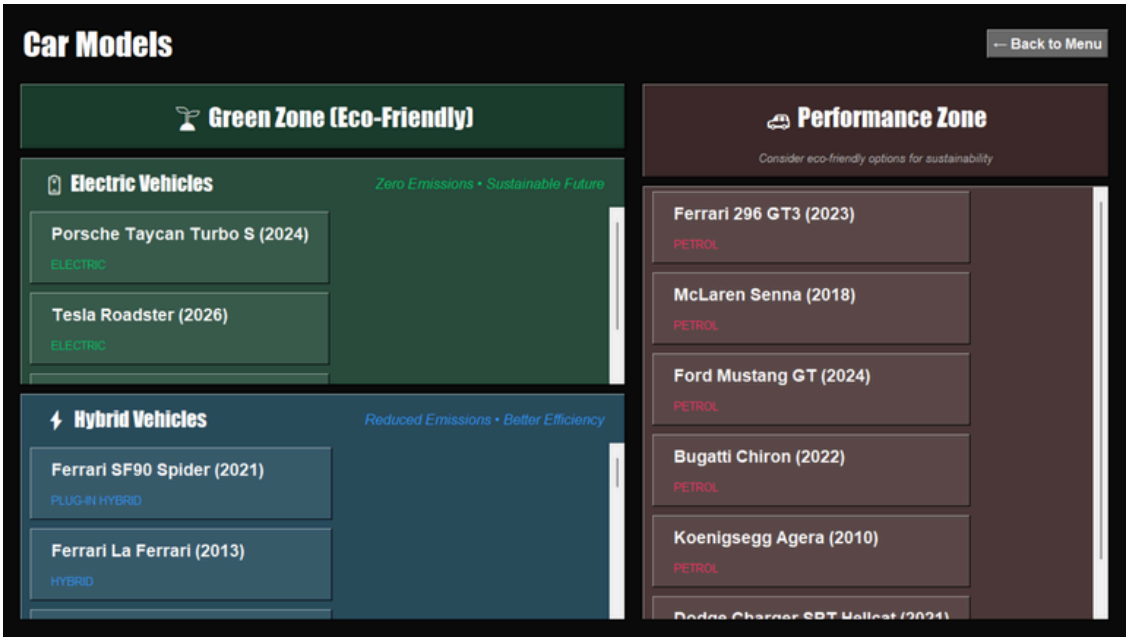
```
        button_grid.columnconfigure(col, weight=1)
        button_grid.rowconfigure(row, weight=1)

        if text == "EXIT":
            btn = Button(btn_frame, text=f"{icon} {text}",
                         font=('Poppins', 12, 'bold'),
                         bg='#dc143c', fg='white',
                         bd=0, pady=12, cursor='hand2',
                         command=command,
                         relief=FLAT, activebackground='#8b0000',
                         activeforeground='white')
        else:
            btn = Button(btn_frame, text=f"{icon} {text}",
                         font=('Poppins', 12, 'bold'),
                         bg='#555555', fg='white',
                         bd=0, pady=12, cursor='hand2',
                         command=command,
                         relief=FLAT, activebackground='#777777',
                         activeforeground='white')

        btn.pack(fill=BOTH, expand=True, padx=5)

        if text != "EXIT":
            def make_hover_func(btn_obj, normal_color, hover_color):
                def on_enter(e):
                    btn_obj.config(bg=hover_color)
                def on_leave(e):
                    btn_obj.config(bg=normal_color)
                return on_enter, on_leave

            enter_func, leave_func = make_hover_func(btn, '#555555', '#777777')
            btn.bind("<Enter>", enter_func)
            btn.bind("<Leave>", leave_func)

    footer_frame = Frame(bg_canvas, bg='#0a0a0a', height=110)
    footer_frame.place(relx=0.5, rely=0.98, anchor="s", relwidth=1)

    Label(footer_frame, text="DRIVE SMART • CHOOSE BOLD • THINK GREEN",
          font=('Orbitron', 15, 'bold'),
          fg='#228B22', bg='#0a0a0a').pack(pady=(15, 5))

    Label(footer_frame, text="Supporting UN SDG 12: Responsible Consumption and Production",
          font=('Roboto', 10, 'italic'),
          fg='#cccccc', bg='#0a0a0a').pack(pady=2)

    Label(footer_frame, text="RevZone © 2025 | Digital Innovation for Sustainable Mobility",
          font=('Roboto', 10),
          fg='#cccccc', bg='#0a0a0a').pack(pady=2)

bg_canvas.create_text(100, screen_height//2, text="🚗
                                                    ",
                      font=('Arial', 30),
                      fill='#dc143c')

    bg_canvas.create_text(screen_width-100, screen_height//2, text="🚙",
                      font=('Arial', 35),
                      fill='#3366cc')

    for i in range(0, screen_width, 50):
        if i % 150 == 0:
            bg_canvas.create_line(i, screen_height-80, i+30, screen_height-50,
                                  fill='#228B22', width=2, dash=(5, 3))

window.bind('<Return>', lambda e: main_menu())
window.mainloop()
```

Output Screen Shots

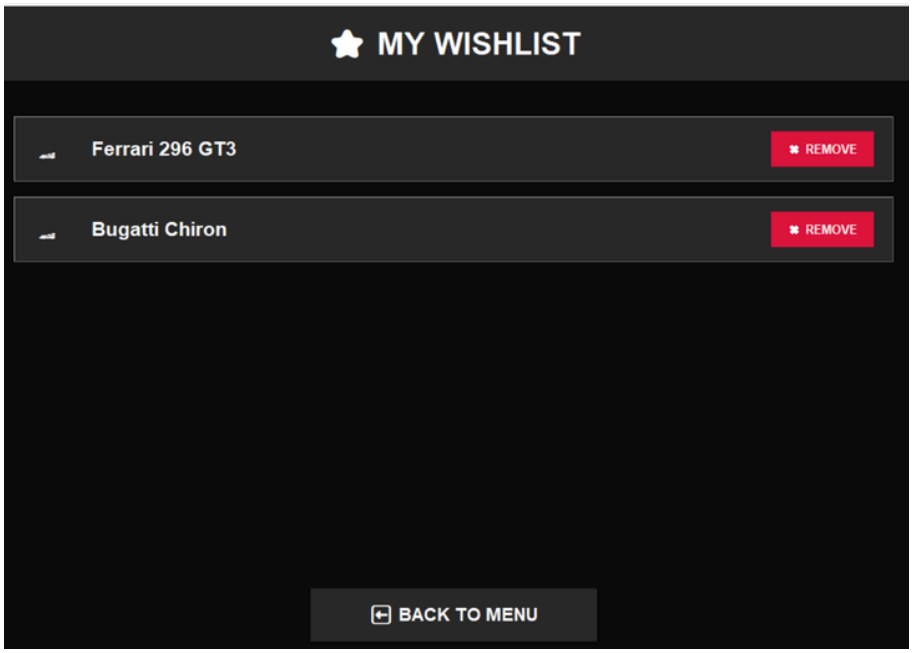
RevZone start screen with animated title.



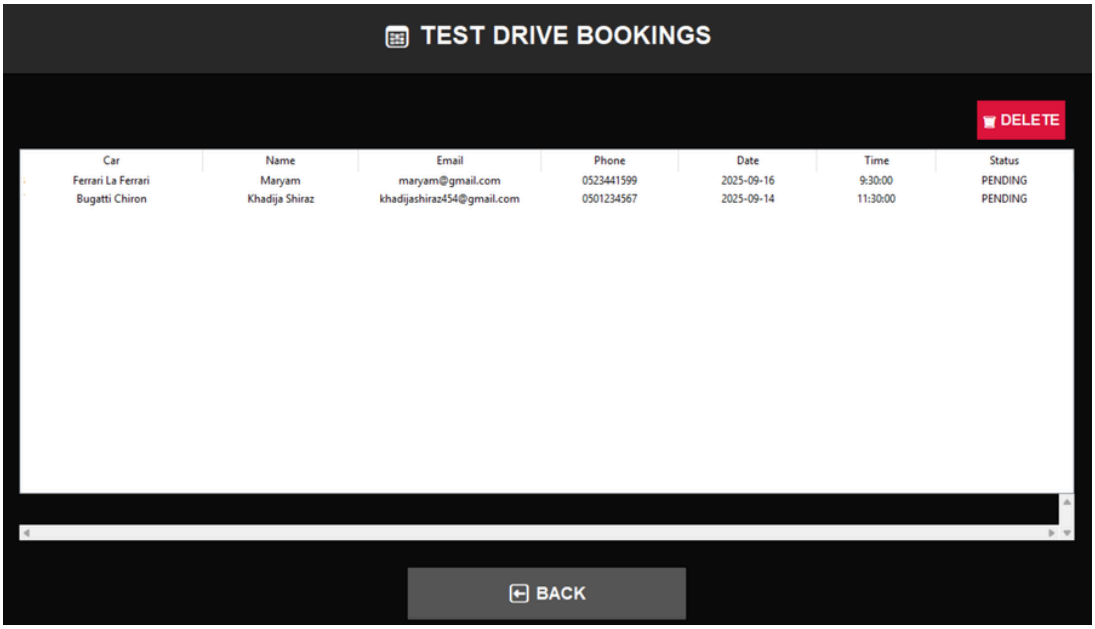
Main menu offering options to browse cars, view wishlist, check test drives, or exit.



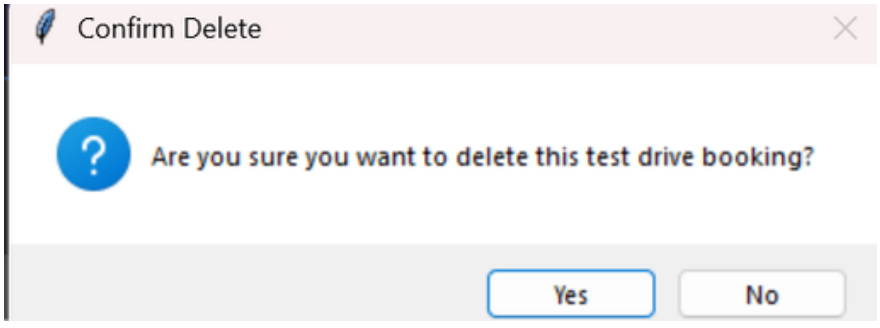
Browse Cars screen - Displaying all available cars, categorized by eco-friendly and performance zones.



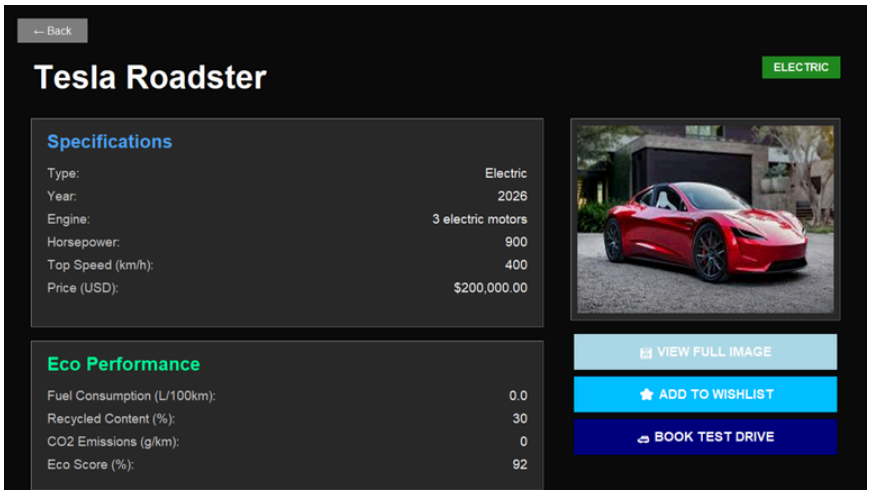
My Wishlist - User wishlist displaying selected cars with option to remove items.



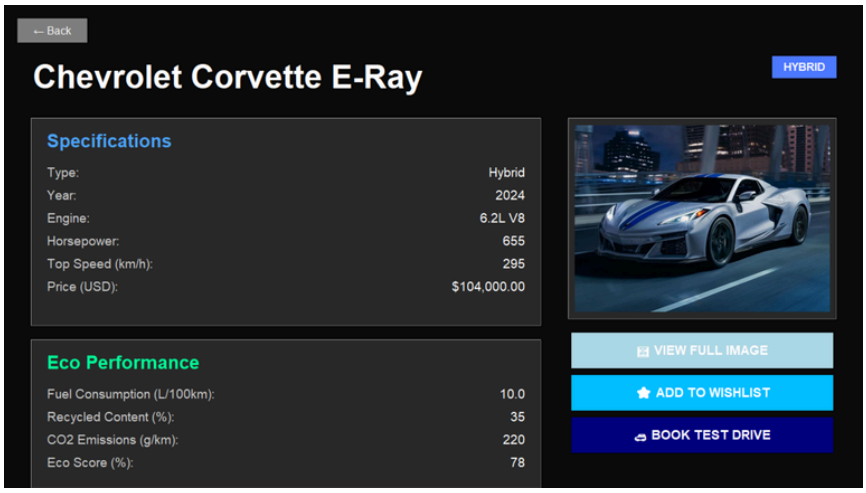
Test Drives - View test drive bookings with an option to delete bookings.



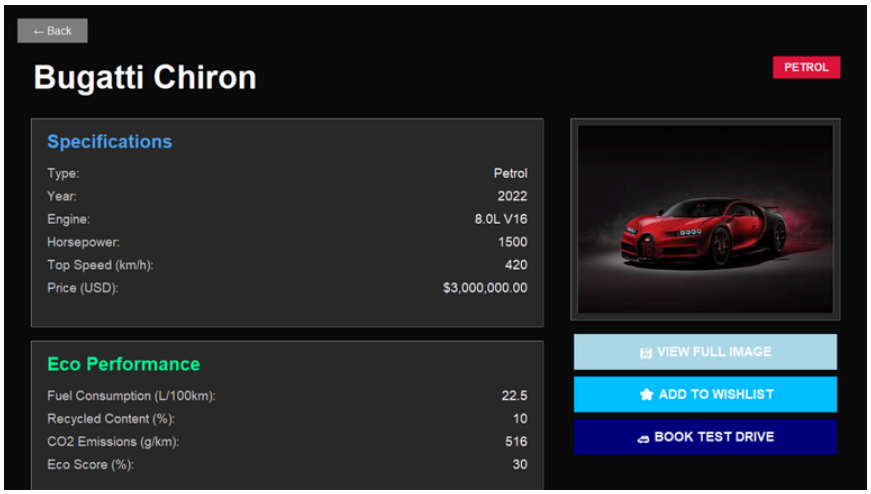
Delete - Message asking for confirmation to delete.



a) Electric model



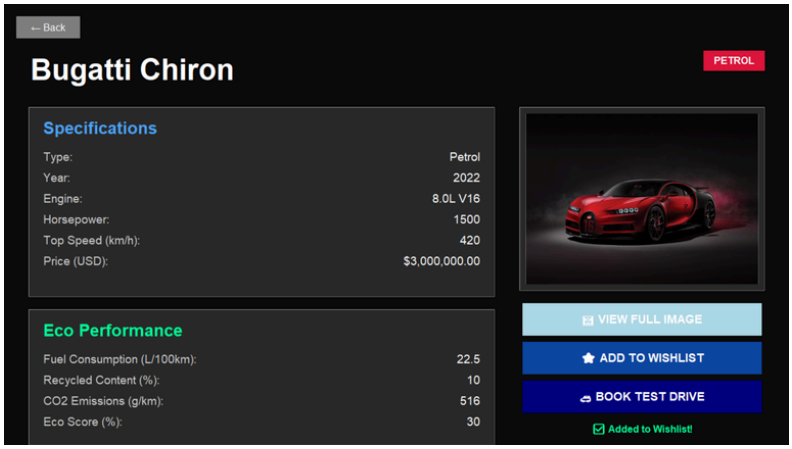
b) Hybrid model



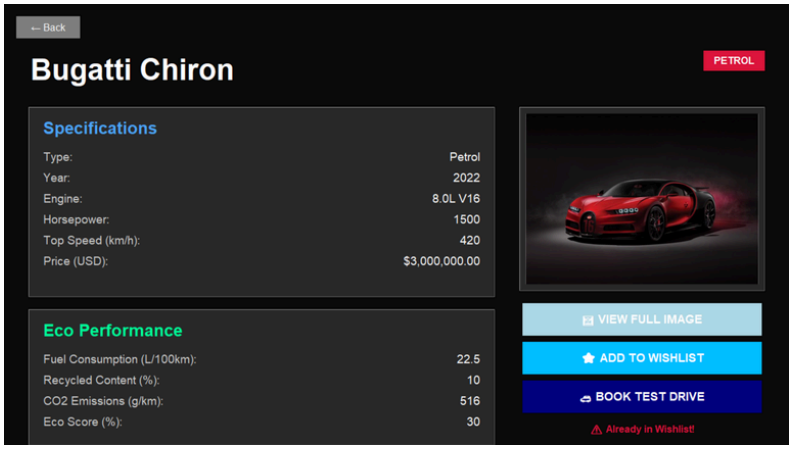
c) Petrol model

Car details screen - with specifications, eco performance, and image preview.

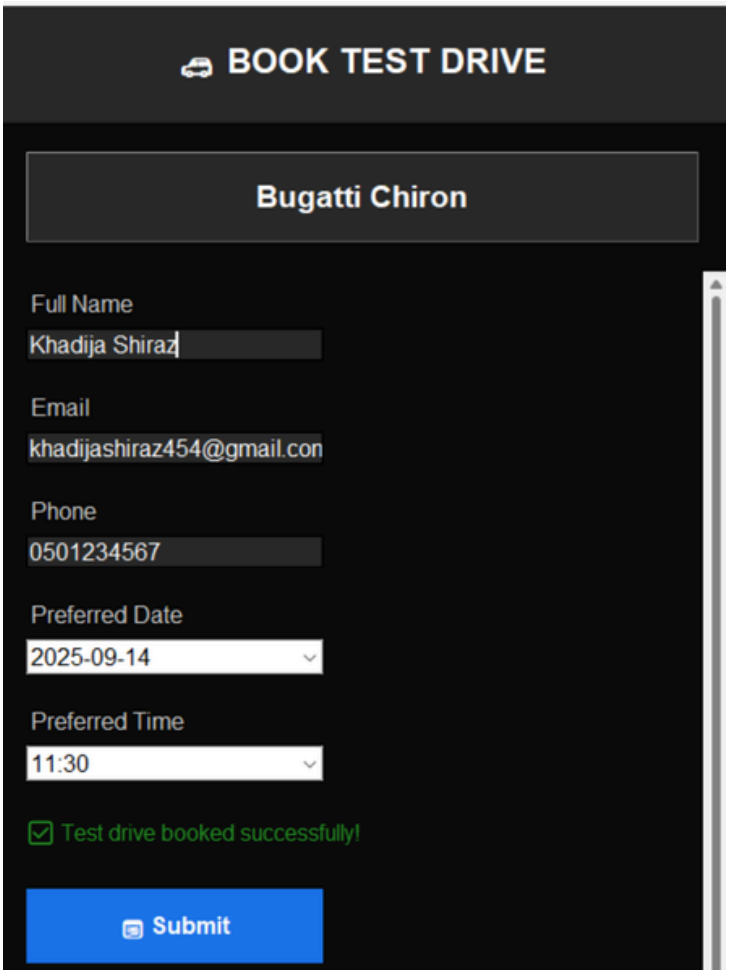
View Full Image - Full-screen view of selected car image.

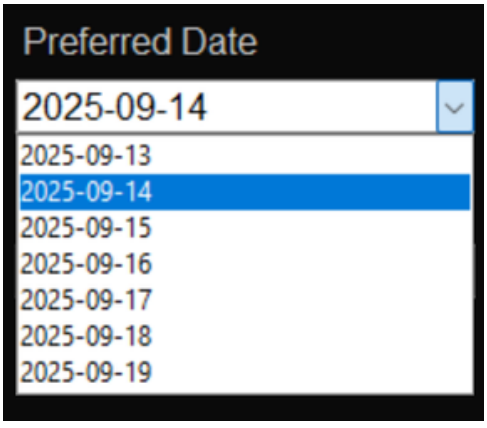


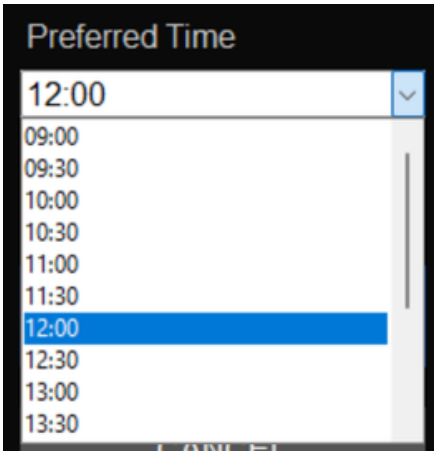
Add to wishlist - Confirmation after adding car.



Add to wishlist - Message when car is already saved.







Book Test Drive - Test drive booking form with date and time selection fields.

Bibliography

- Python 3 Documentation – https://docs.python.org/3/
- Tkinter GUI Library Documentation – https://docs.python.org/3/library/tkinter.html
- MySQL Official Documentation – https://dev.mysql.com/doc/
- https://sdgs.un.org/goals/goal12
- https://www.w3schools.com/
- https://www.tutorialspoint.com/